

✓ Practical: Uninform and Informed Search Algorithms

Name: Samarth Kadam

PRN: 202401040131

Batch: A2

Roll No. 33

Emergency Ambulance Route Finder Using Search Algorithms

This practical implements a real-life Emergency Ambulance Route Finder in which an ambulance must find a route from a Hospital to a Patient using different Artificial Intelligence search techniques. The program contains five uninformed search algorithms and two informed search algorithms. The code uses a weighted city-road graph and heuristic values for the informed methods.

1. Aim

To design and implement an Emergency Ambulance Route Finder using various Artificial Intelligence search algorithms such as BFS, DFS, DLS, IDS, Bidirectional Search, Greedy Best-First Search, and A* Search, and compare their performance in finding a route from a hospital to a patient.

2. Objectives

The objectives of this practical are:

To understand the concept of state-space search in Artificial Intelligence. To implement different uninformed search algorithms. To implement informed search algorithms using heuristic information. To find a route between a starting location and a destination. To calculate the path cost of the discovered route. To compare the number of nodes visited by different algorithms. To understand the difference between informed and uninformed search. To identify an appropriate search algorithm for route-finding applications.

3. Application Details

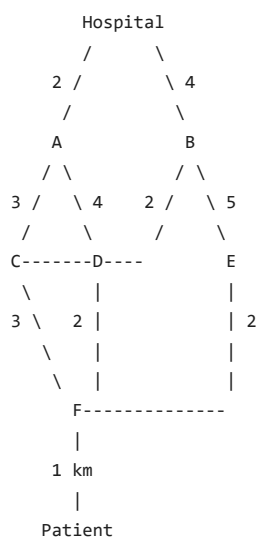
Application Name

Emergency Ambulance Route Finder

Real-Life Problem

Consider a city in which an ambulance is located at a hospital and has to reach a patient during an emergency.

The city contains several road junctions:



Each road has a distance/cost associated with it.

For example:

Hospital → A = 2 km A → C = 3 km C → F = 3 km F → Patient = 1 km

Therefore:

Hospital → A → C → F → Patient

has a total cost of:

$$2 + 3 + 3 + 1 = 9 \text{ km}$$

The program stores these locations and distances in a graph.

4. Algorithms Used

The practical contains 7 search algorithms.

A. Uninformed Search Algorithms

These algorithms do not use heuristic information.

1. Breadth First Search — BFS

BFS explores nodes level by level.

```
START
|
Level 0
|
Level 1
|
Level 2
|
GOAL
```

1. Insert Start into Queue.
2. Mark Start as visited.
3. While Queue is not empty: a. Remove a node from the front. b. If node is Goal: return path. c. Add unvisited neighboring nodes to Queue.
4. If Queue becomes empty: return failure.

The implementation uses Python's deque as the queue.

2. Depth First Search — DFS **bold text** DFS explores one branch as deeply as possible before backtracking.

Principle Uses a Stack Follows LIFO Goes deep before exploring other branches Data structure Stack Basic algorithm DFS(Start, Goal)

1. Insert Start into Stack.
2. While Stack is not empty: a. Remove the top node. b. Mark it as visited. c. If node is Goal: return path. d. Push unvisited neighboring nodes.
3. If Stack becomes empty: return failure.

The program implements DFS using a Python list as a stack.

3. Depth Limited Search — DLS

DLS is a modified version of DFS in which a maximum depth is specified.

For example:

Depth Limit = 3

The algorithm cannot search beyond depth 3.

Principle DLS = DFS + Depth Limit Basic algorithm DLS(Start, Goal, Limit)

1. Start at the initial node.
2. Search using DFS.
3. Keep track of current depth.
4. If Goal is found: return path.
5. If depth reaches Limit: stop searching that branch.
6. Continue with other branches.

The program accepts the depth limit dynamically.

4. Iterative Deepening Search — IDS

IDS repeatedly performs DLS with increasing depth limits.

```
DLS depth 0
↓
DLS depth 1
↓
DLS depth 2
↓
```

```
DLS depth 3
  ↓
...
Basic algorithm
IDS(Start, Goal)
```

1. Set depth = 0.
2. Perform DLS(Start, Goal, depth).
3. If Goal is found: return path.
4. Increase depth.
5. Repeat until Goal is found.

The program repeatedly calls DLS until the destination is found.

5. Bidirectional Search

Bidirectional Search searches simultaneously from:

```
Hospital → → →
           ← ← ← Patient

The two searches continue until they meet.

Principle
Forward Search : Hospital → ...
Backward Search : Patient → ...
           ↓
           Meeting
```

Basic algorithm

1. Start one BFS from Start.
2. Start another BFS from Goal.
3. Expand both searches.
4. Check whether their visited nodes intersect.
5. When they meet: construct the complete path.

The program maintains separate forward and backward queues and parent relationships.

B. Informed Search Algorithms

These algorithms use heuristic information.

The heuristic values in the program represent estimated distance to the patient.

6. Greedy Best-First Search

Greedy Best-First Search chooses the node that appears closest to the destination.

Formula $f(n) = h(n)$

Where:

$h(n)$ = estimated cost from current node to goal

For example:

$A \rightarrow C \quad h(C) = 4 \quad A \rightarrow D \quad h(D) = 4$

The algorithm selects a node according to its heuristic value.

Basic algorithm Greedy Best First Search

1. Put Start in Priority Queue.
2. Calculate $h(n)$.
3. Select the node with smallest $h(n)$.
4. Expand the node.
5. Repeat until Goal is reached.

The implementation uses heapq as a priority queue.

7. A-star Search

A* combines:

actual distance already travelled estimated distance remaining Formula $f(n) = g(n) + h(n)$

Where:

$g(n)$ = actual cost from Start to n $h(n)$ = estimated cost from n to Goal $f(n)$ = total estimated cost

For example:

Hospital $\rightarrow$ A

$g(A) = 2 \text{ km}$ $h(A) = 6 \text{ km}$

$f(A) = 2 + 6 = 8$

The node with the smallest $f(n)$ is selected.

Basic algorithm A*(Start, Goal)

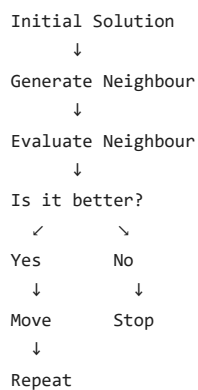
1. Put Start in Priority Queue.
2. Set $g(\text{Start}) = 0$.
3. Calculate $f(n) = g(n) + h(n)$.
4. Select node with smallest $f(n)$.
5. Expand that node.
6. Update costs of neighboring nodes.
7. Repeat until Goal is reached.

The program maintains the best known cost and calculates $f = g + h$.

8. Hill Climbing Algorithm

Hill Climbing is a local search algorithm used for solving optimization problems. It starts with an initial solution and repeatedly moves toward a better neighboring solution.

The name comes from the idea of climbing a hill: at every step, we try to move upward toward a better position until no better position is available.



In our application:

Initial state: A valid route from Hospital to Patient. Neighbor: Another valid route that differs from the current route. Evaluation: Total route distance. Objective: Minimize the ambulance travel distance.

Example:

Route 1 = Hospital $\rightarrow$ B $\rightarrow$ E $\rightarrow$ I $\rightarrow$ K $\rightarrow$ L $\rightarrow$ M $\rightarrow$ N $\rightarrow$ Patient
Cost = 33 km

$\downarrow$ improvement

Route 2 = Hospital $\rightarrow$ A $\rightarrow$ E $\rightarrow$ I $\rightarrow$ K $\rightarrow$ L $\rightarrow$ M $\rightarrow$ N $\rightarrow$ Patient
Cost = 31 km

$\downarrow$ improvement

Route 3 = Hospital $\rightarrow$ A $\rightarrow$ D $\rightarrow$ H $\rightarrow$ I $\rightarrow$ K $\rightarrow$ L $\rightarrow$ M $\rightarrow$ N $\rightarrow$ Patient
Cost = 29 km

The algorithm continues until it cannot find a better neighboring route.

Basic Components of Hill Climbing 1. Initial State

The algorithm starts with a candidate solution.

2. Neighbor Generation

Small changes are made to produce neighboring solutions.

3. Evaluation Function

The quality of each solution is measured.

For your route problem:

Cost = Total Distance

4. Selection

A better neighbor is selected.

5. Termination

The algorithm stops when no better neighbor exists or a stopping condition is reached.

Advantages of Hill Climbing Simple to understand and implement. Uses relatively little memory. Can improve a solution quickly. Useful for optimization problems. Easy to adapt to real-life problems. Limitations Can get stuck at local maxima. May stop at a plateau. Does not guarantee the global optimum. Result can depend on the initial solution.

9. Genetic Algorithm

A Genetic Algorithm (GA) is an optimization algorithm inspired by natural evolution and the principle of survival of better solutions.

Instead of maintaining only one solution, Genetic Algorithm maintains a population of candidate solutions and evolves them over several generations.

The basic process is:

```
Population
↓
Fitness Evaluation
↓
Selection
↓
Crossover
↓
Mutation
↓
New Population
↓
Repeat
```

Genetic Algorithm for Ambulance Route Finding

In your same problem, each possible route from:

Hospital → Patient

is treated as a candidate solution.

Important Terms in Genetic Algorithm

1. Population

A collection of candidate solutions.

Population:

Route 1 Route 2 Route 3 Route 4 Route 5 ... 2. Chromosome

One candidate solution is called a chromosome.

For example:

Hospital → A → E → I → K → L → M → N → Patient

This represents one possible ambulance route.

3. Gene

A gene represents one element of a chromosome.

For example:

Hospital → A → E → I → K → L → M → N → Patient ↑ Gene

In route-based problems, the genes represent locations/junctions.

4. Fitness Function

The fitness function measures how good a solution is.

Since the ambulance wants a shorter route, lower distance means a better solution.

A simple fitness function is:

$$\text{Fitness} = 1 / (\text{Distance} + 1)$$

Therefore:

$$\text{Distance} = 20 \text{ km} \quad \text{Fitness} = 1 / 21$$

$$\text{Distance} = 40 \text{ km} \quad \text{Fitness} = 1 / 41$$

So the 20 km route has higher fitness.

5. Selection

Selection chooses the better solutions to become parents for the next generation.

Example:

Route A → 40 km Route B → 35 km Route C → 28 km Route D → 45 km

Better routes such as B and C are more likely to be selected.

6. Crossover

Crossover combines information from two parent solutions to produce a new solution.

A route crossover operation can combine useful parts of these parents to generate a child.

For route problems, the crossover method must ensure that the resulting route contains valid road connections.

7. Mutation

Mutation makes a small random change to a solution.

Mutation introduces diversity into the population and can help the algorithm explore routes that were not already present.

8. Generation

One complete cycle of:

Selection ↓ Crossover ↓ Mutation

produces a new generation.

For example:

Generation 1 → Best = 40 km Generation 2 → Best = 35 km Generation 3 → Best = 32 km Generation 4 → Best = 29 km

The goal is to improve the best solution over generations.

Genetic Algorithm Steps 1. Generate an initial population.

2. Calculate the fitness of every solution.
3. Select the better solutions.
4. Select parents.
5. Perform crossover.
6. Perform mutation.
7. Create a new population.
8. Evaluate the new population.
9. Repeat for a specified number of generations.
10. Return the best solution found.

Genetic Algorithm Flow START ↓ Generate Population ↓ Calculate Fitness ↓ Selection ↓ Crossover ↓ Mutation ↓ New Population ↓ Termination? /

No Yes ↓ ↓ Repeat STOP

Advantages of Genetic Algorithm

Works with multiple candidate solutions. Can explore a large search space. Useful for complex optimization problems. Mutation provides additional exploration. Suitable for routing, scheduling, allocation, and design problems. Does not require exploring every possible solution.

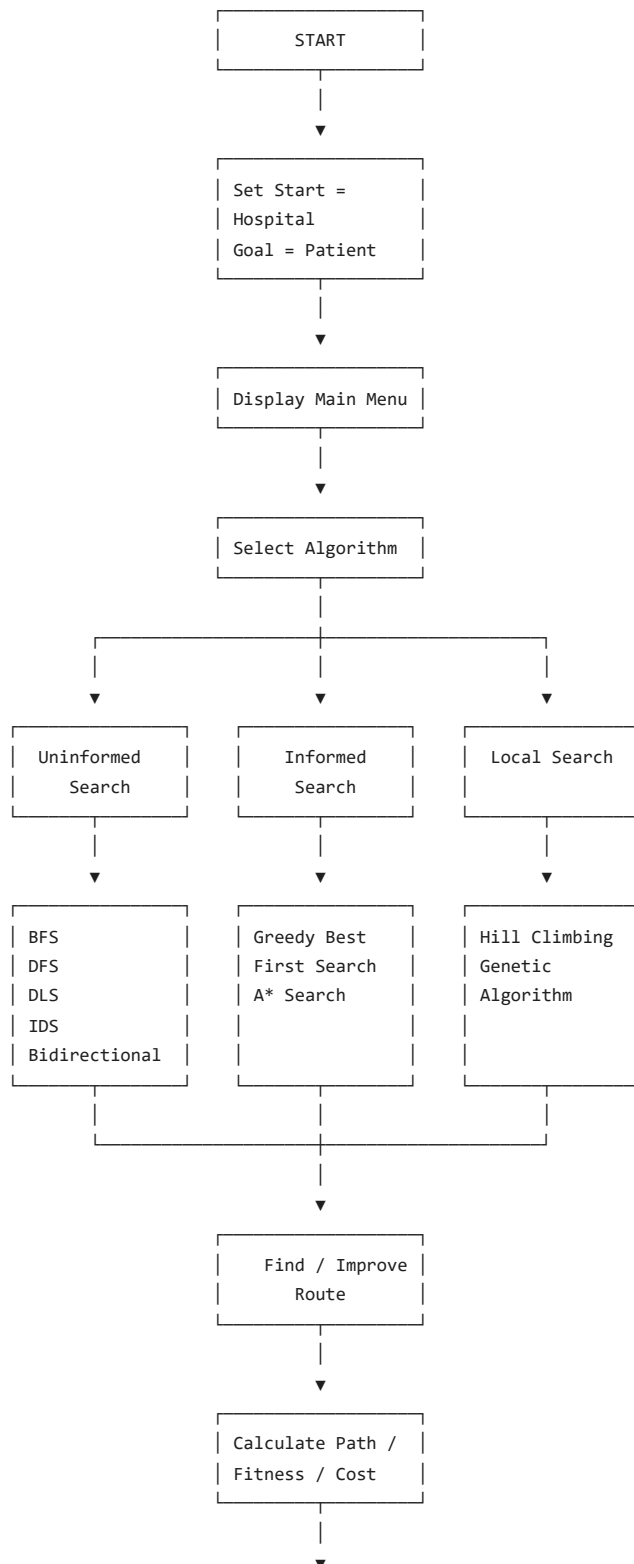
Limitations

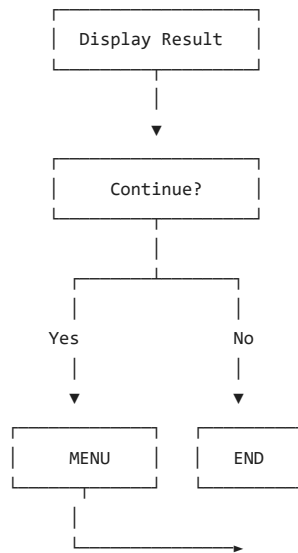
Can require many generations. Results can depend on parameters. Uses randomness, so results may vary. Does not necessarily guarantee the global optimum. Crossover and mutation need to be designed appropriately for the problem.

5. Difference Between Uninformed and Informed Search

Uninformed Search	Informed Search	Does not use heuristic information	Uses heuristic information	Has no knowledge of how close a node is to goal
Estimates distance to goal	Examples: BFS, DFS, DLS, IDS	Examples: Greedy, A*	Searches based on graph structure	Searches based on graph + heuristic
May explore more unnecessary nodes	Can guide search toward goal			

You can draw the following flowchart in your practical file:





7. Detailed Program

The main program uses the city graph, heuristic values, seven search functions, and the GUI. The graph stores the road distances, while the heuristic dictionary is used by Greedy Best-First and A*.

The program creates an interactive Emergency Ambulance Route Finder. The user can select any search algorithm and find a route between Hospital and Patient. The program displays the discovered path, path cost, and visited nodes. It also provides a comparison of all algorithms and displays the city road map.

The complete program you developed contains the seven algorithms and the route-finding GUI. The original implementation also includes a comparison function and city-map display.

8. Observation

After executing the program, observe the following:

BFS

BFS searches the graph level by level and generally finds a path with the minimum number of edges.

DFS

DFS goes deeper into one branch before backtracking.

DLS

The result depends on the specified depth limit.

For example:

Depth Limit = 2

may not be sufficient to reach the patient.

A larger depth allows the search to reach deeper nodes.

IDS

IDS gradually increases the depth:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots$

until a solution is found.

Bidirectional Search

The search progresses simultaneously from the hospital and patient sides.

Greedy Best-First Search

Greedy uses only:

$h(n)$

and attempts to move toward the apparently closest node.

A*

A* uses:

$f(n) = g(n) + h(n)$

so it considers both the route already travelled and the estimated remaining distance.

9. Result Analysis

For this particular road network, the important route is:

Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient

Its distance is:

Hospital $\rightarrow$ A = 2 km A $\rightarrow$ C = 3 km C $\rightarrow$ F = 3 km F $\rightarrow$ Patient = 1 km

Therefore:

Total = 2 + 3 + 3 + 1 = 9 km Expected comparison Algorithm Category Expected Path Cost BFS Uninformed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km DFS Uninformed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km DLS Uninformed Depends on depth limit 9 km when sufficient IDS Uninformed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km Bidirectional Uninformed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km Greedy Best First Informed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km A* Informed Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient 9 km

The exact visited-node count/order is implementation-dependent, so it should be copied from the output generated by your executed program rather than manually assumed.

10. Result

The Emergency Ambulance Route Finder was successfully implemented using BFS, DFS, DLS, IDS, Bidirectional Search, Greedy Best-First Search, and A* Search. The application successfully finds a route from the Hospital to the Patient, calculates the route cost, displays visited nodes, and compares the performance of different search algorithms. For the given road network, the minimum route distance is 9 km through Hospital $\rightarrow$ A $\rightarrow$ C $\rightarrow$ F $\rightarrow$ Patient.

11. Conclusion

This practical demonstrates how Artificial Intelligence search algorithms can be applied to a real-life emergency route-finding problem. Uninformed algorithms search without heuristic knowledge, whereas informed algorithms use heuristic information to guide the search. Greedy Best-First Search uses only the estimated remaining cost, while A* combines the actual cost and estimated cost. The practical shows that the choice of search algorithm affects the number of nodes explored, search behavior, and efficiency of route finding.

```
# =====
# 🚑 EMERGENCY AMBULANCE ROUTE FINDER
# =====
# GOOGLE COLAB GUI
#
# UNINFORMED SEARCH:
# 1. BFS
# 2. DFS
# 3. DLS
# 4. IDS
# 5. Bidirectional Search
#
# INFORMED SEARCH:
# 6. Greedy Best First Search
# 7. A* Search
#
# LOCAL SEARCH:
# 8. Hill Climbing
# 9. Genetic Algorithm
#
# OTHER:
# 10. Compare All Algorithms
# 11. Display City Map
# 12. Exit
# =====

# =====
# IMPORTS
# =====

from collections import deque
import heapq
import random
import ipywidgets as widgets
from IPython.display import display, clear_output

# =====
# START AND GOAL
# =====
```

```
START = "Hospital"
GOAL = "Patient"
```

```
# =====
# CITY ROAD MAP
# =====
```

```
graph = {

    "Hospital": {
        "A": 4,
        "B": 6,
        "C": 8
    },

    "A": {
        "Hospital": 4,
        "D": 3,
        "E": 6
    },

    "B": {
        "Hospital": 6,
        "E": 2,
        "F": 5
    },

    "C": {
        "Hospital": 8,
        "F": 3,
        "G": 6
    },

    "D": {
        "A": 3,
        "E": 2,
        "H": 5
    },

    "E": {
        "A": 6,
        "B": 2,
        "D": 2,
        "F": 3,
        "H": 4,
        "I": 6
    },

    "F": {
        "B": 5,
        "C": 3,
        "E": 3,
        "G": 2,
        "I": 4
    },

    "G": {
        "C": 6,
        "F": 2,
        "I": 2,
        "J": 5
    },

    "H": {
        "D": 5,
        "E": 4,
        "I": 3,
        "K": 6
    },

    "I": {
        "E": 6,
        "F": 4,
        "G": 2,
        "H": 3,
        "J": 2,
        "K": 4
    },

    "J": {
        "G": 5
```

```

        "I": 2,
        "K": 2,
        "L": 5
    },

    "K": {
        "H": 6,
        "I": 4,
        "J": 2,
        "L": 3
    },

    "L": {
        "J": 5,
        "K": 3,
        "M": 2
    },

    "M": {
        "L": 2,
        "N": 3
    },

    "N": {
        "M": 3,
        "Patient": 4
    },

    "Patient": {
        "N": 4
    }
}

# =====
# HEURISTIC VALUES
# =====

heuristic = {

    "Hospital": 30,
    "A": 26,
    "B": 24,
    "C": 23,
    "D": 23,
    "E": 22,
    "F": 20,
    "G": 18,
    "H": 18,
    "I": 16,
    "J": 14,
    "K": 12,
    "L": 9,
    "M": 7,
    "N": 4,
    "Patient": 0
}

# =====
# PATH COST
# =====

def path_cost(path):

    if not path:
        return 0

    cost = 0

    for i in range(len(path) - 1):

        current = path[i]
        next_node = path[i + 1]

        cost += graph[current][next_node]

    return cost

# =====
# PATH VALIDATION

```

```

# =====
def is_valid_path(path):

    if not path:
        return False

    if path[0] != START:
        return False

    if path[-1] != GOAL:
        return False

    for i in range(len(path) - 1):

        current = path[i]
        next_node = path[i + 1]

        if current not in graph:
            return False

        if next_node not in graph[current]:
            return False

    # No repeated nodes
    if len(path) != len(set(path)):
        return False

    return True

# =====
# SAFE PATH COST
# =====

def safe_path_cost(path):

    if not is_valid_path(path):

        return float("inf")

    return path_cost(path)

# =====
# 1. BFS
# UNINFORMED SEARCH
# =====

def bfs(start, goal):

    queue = deque([
        (start, [start])
    ])

    visited = set()

    visited_order = []

    while queue:

        current, path = queue.popleft()

        if current in visited:
            continue

        visited.add(current)

        visited_order.append(current)

        if current == goal:

            return path, visited_order

        for neighbor in graph[current]:

            if neighbor not in visited:

                queue.append(
                    (
                        neighbor,
                        path + [neighbor]
                    )

```

```

    )

    return None, visited_order

# =====
# 2. DFS
# UNINFORMED SEARCH
# =====

def dfs(start, goal):

    stack = [
        (start, [start])
    ]

    visited = set()

    visited_order = []

    while stack:

        current, path = stack.pop()

        if current in visited:
            continue

        visited.add(current)

        visited_order.append(current)

        if current == goal:

            return path, visited_order

        neighbors = list(
            graph[current].keys()
        )

        neighbors.reverse()

        for neighbor in neighbors:

            if neighbor not in visited:

                stack.append(
                    (
                        neighbor,
                        path + [neighbor]
                    )
                )

    return None, visited_order

# =====
# 3. DLS
# UNINFORMED SEARCH
# =====

def dls(start, goal, limit):

    visited_order = []

    def search(current, path, depth):

        visited_order.append(current)

        if current == goal:

            return path

        if depth >= limit:

            return None

        for neighbor in graph[current]:

            if neighbor not in path:

                result = search(
                    neighbor,
                    path + [neighbor]

```

```

        depth + 1
    )

    if result is not None:

        return result

    return None

path = search(
    start,
    [start],
    0
)

return path, visited_order

# =====
# 4. IDS
# UNINFORMED SEARCH
# =====

def ids(start, goal):

    complete_order = []

    for depth in range(20):

        path, visited = dls(
            start,
            goal,
            depth
        )

        complete_order.extend(
            visited
        )

        if path is not None:

            return path, complete_order

    return None, complete_order

# =====
# 5. BIDIRECTIONAL SEARCH
# UNINFORMED SEARCH
# =====

def bidirectional_search(start, goal):

    if start == goal:

        return [start], [start]

    forward_queue = deque([start])

    backward_queue = deque([goal])

    forward_parent = {
        start: None
    }

    backward_parent = {
        goal: None
    }

    visited_order = []

    meeting_node = None

    while forward_queue and backward_queue:

        # -----
        # FORWARD SEARCH
        # -----

        current = forward_queue.popleft()

```

```

        if current not in visited_order:

            visited_order.append(current)

        for neighbor in graph[current]:

            if neighbor not in forward_parent:

                forward_parent[neighbor] = current

                forward_queue.append(
                    neighbor
                )

                if neighbor in backward_parent:

                    meeting_node = neighbor

                    break

            if meeting_node is not None:

                break

        # -----
        # BACKWARD SEARCH
        # -----

        current = backward_queue.popleft()

        if current not in visited_order:

            visited_order.append(current)

        for neighbor in graph[current]:

            if neighbor not in backward_parent:

                backward_parent[neighbor] = current

                backward_queue.append(
                    neighbor
                )

                if neighbor in forward_parent:

                    meeting_node = neighbor

                    break

            if meeting_node is not None:

                break

        if meeting_node is None:

            return None, visited_order

        # -----
        # Hospital -> Meeting
        # -----

        forward_path = []

        current = meeting_node

        while current is not None:

            forward_path.append(current)

            current = forward_parent[current]

        forward_path.reverse()

        # -----
        # Meeting -> Patient
        # -----

        backward_path = []

```

```

        current = backward_parent[
            meeting_node
        ]

    while current is not None:

        backward_path.append(current)

        current = backward_parent[current]

    final_path = (
        forward_path +
        backward_path
    )

    return final_path, visited_order

# =====
# 6. GREEDY BEST FIRST SEARCH
# INFORMED SEARCH
# =====

def greedy_best_first_search(start, goal):

    priority_queue = []

    heapq.heappush(
        priority_queue,
        (
            heuristic[start],
            start,
            [start]
        )
    )

    visited = set()

    visited_order = []

    while priority_queue:

        h, current, path = heapq.heappop(
            priority_queue
        )

        if current in visited:

            continue

        visited.add(current)

        visited_order.append(current)

        if current == goal:

            return path, visited_order

        for neighbor in graph[current]:

            if neighbor not in visited:

                heapq.heappush(
                    priority_queue,
                    (
                        heuristic[neighbor],
                        neighbor,
                        path + [neighbor]
                    )
                )

    return None, visited_order

# =====
# 7. A* SEARCH
# INFORMED SEARCH
# =====

def a_star(start, goal):

```

```

priority_queue = []

heapq.heappush(
    priority_queue,
    (
        heuristic[start],
        0,
        start,
        [start]
    )
)

best_cost = {
    start: 0
}

visited = set()

visited_order = []

while priority_queue:

    f, g, current, path = heapq.heappop(
        priority_queue
    )

    if current in visited:

        continue

    visited.add(current)

    visited_order.append(current)

    if current == goal:

        return path, visited_order

    for neighbor, distance in graph[current].items():

        new_g = g + distance

        if (
            neighbor not in best_cost
            or new_g < best_cost[neighbor]
        ):

            best_cost[neighbor] = new_g

            new_f = (
                new_g +
                heuristic[neighbor]
            )

            heapq.heappush(
                priority_queue,
                (
                    new_f,
                    new_g,
                    neighbor,
                    path + [neighbor]
                )
            )

    return None, visited_order

# =====
# FIND ALL SIMPLE VALID PATHS
# =====

def all_simple_paths(
    current,
    goal,
    path=None
):

    if path is None:

        path = [current]

    if current == goal:

```

```

        return [path]

    paths = []

    for neighbor in graph[current]:

        if neighbor not in path:

            paths.extend(
                all_simple_paths(
                    neighbor,
                    goal,
                    path + [neighbor]
                )
            )

    return paths

# =====
# PRE-COMPUTE ALL VALID ROUTES
# =====

ALL_VALID_PATHS = all_simple_paths(
    START,
    GOAL
)

# =====
# HILL CLIMBING
# LOCAL SEARCH
# =====

def hill_climbing(start, goal):

    valid_paths = [
        p
        for p in ALL_VALID_PATHS
        if p[0] == start and p[-1] == goal
    ]

    if not valid_paths:

        return None, []

    # -----
    # Start with a long route.
    # -----

    current = max(
        valid_paths,
        key=safe_path_cost
    )

    current_cost = safe_path_cost(
        current
    )

    history = [
        current.copy()
    ]

    # -----
    # Hill Climbing
    # -----

    for _ in range(50):

        better_neighbors = []

        # A neighbouring route shares the starting
        # portion of the current route.

        for candidate in valid_paths:

            if candidate == current:

                continue

```

```

        common_prefix = 0

        for a, b in zip(
            current,
            candidate
        ):

            if a == b:

                common_prefix += 1

            else:

                break

        # At least START + one junction
        # should remain the same.

        if common_prefix >= 2:

            candidate_cost = safe_path_cost(
                candidate
            )

            if candidate_cost < current_cost:

                better_neighbors.append(
                    candidate
                )

        if not better_neighbors:

            break

        # Select best neighbour

        best_neighbor = min(
            better_neighbors,
            key=safe_path_cost
        )

        # Move to better neighbour

        current = best_neighbor.copy()

        current_cost = safe_path_cost(
            current
        )

        history.append(
            current.copy()
        )

    return current, history

# =====
# GENETIC ALGORITHM
# =====

def route_fitness(route):

    cost = safe_path_cost(
        route
    )

    if cost == float("inf"):

        return 0

    return 1 / (
        cost + 1
    )

# =====
# CREATE RANDOM ROUTE
# =====

```

```

def create_random_route(rng):

    return rng.choice(
        ALL_VALID_PATHS
    ).copy()

# =====
# CREATE POPULATION
# =====

def create_population(
    size,
    rng
):

    population = []

    for _ in range(size):

        population.append(
            create_random_route(rng)
        )

    return population

# =====
# SELECTION
# =====

def select_parents(
    population
):

    sorted_population = sorted(
        population,
        key=route_fitness,
        reverse=True
    )

    parent_count = max(
        2,
        len(population) // 2
    )

    return sorted_population[
        :parent_count
    ]

# =====
# VALIDATE CROSSOVER RESULT
# =====

def valid_route(route):

    return is_valid_path(route)

# =====
# CROSSOVER
# =====

def crossover(
    parent1,
    parent2,
    rng
):

    # Find common intermediate nodes

    common_nodes = [

        node

        for node in parent1[1:-1]

        if node in parent2[1:-1]

    ]

```

```

# No common point
# Return one parent

if not common_nodes:

    return rng.choice(
        [
            parent1,
            parent2
        ]
    ).copy()

# Random common point

meeting = rng.choice(
    common_nodes
)

index1 = parent1.index(
    meeting
)

index2 = parent2.index(
    meeting
)

# Prefix of parent 1
# + suffix of parent 2

child = (
    parent1[:index1]
    + parent2[index2:]
)

# Check validity

if valid_route(child):

    return child

# Otherwise use parent

return rng.choice(
    [
        parent1,
        parent2
    ]
).copy()

# =====
# MUTATION
# =====

def mutation(
    route,
    rng,
    mutation_rate=0.20
):

    new_route = route.copy()

    if rng.random() < mutation_rate:

        # Select any other valid route

        new_route = rng.choice(
            ALL_VALID_PATHS
        ).copy()

    return new_route

# =====
# GENETIC ALGORITHM

```

```

# =====
def genetic_algorithm(
    start="Hospital",
    goal="Patient",
    population_size=20,
    generations=40,
    mutation_rate=0.20,
    seed=42
):
    rng = random.Random(seed)

    # -----
    # Initial Population
    # -----

    population = create_population(
        population_size,
        rng
    )

    if not population:
        return None, float("inf"), []

    # -----
    # Best solution
    # -----

    best_route = min(
        population,
        key=safe_path_cost
    ).copy()

    best_cost = safe_path_cost(
        best_route
    )

    progress = []

    # -----
    # GENERATIONS
    # -----

    for generation in range(
        generations
    ):

        # Current best

        current_best = min(
            population,
            key=safe_path_cost
        )

        current_cost = safe_path_cost(
            current_best
        )

        # Update global best

        if current_cost < best_cost:

            best_cost = current_cost

            best_route = current_best.copy()

        # Save progress

        progress.append(
            (
                generation + 1,
                best_route.copy(),
                best_cost
            )
        )

```

```

    )
)

# -----
# Selection
# -----

parents = select_parents(
    population
)

# -----
# New Population
# -----

new_population = [

    best_route.copy()

]

# -----
# Crossover + Mutation
# -----

while len(new_population) < population_size:

    parent1, parent2 = rng.sample(
        parents,
        2
    )

    child = crossover(
        parent1,
        parent2,
        rng
    )

    child = mutation(
        child,
        rng,
        mutation_rate
    )

    # Safety validation

    if valid_route(child):

        new_population.append(
            child
        )

    else:

        new_population.append(
            parent1.copy()
        )

    population = new_population

return (
    best_route,
    best_cost,
    progress
)

# =====
# OUTPUT AREA
# =====

output = widgets.Output()

```

```

# =====
# TITLE
# =====

title = widgets.HTML(
    value=""

    <div style="
        text-align:center;
        font-size:28px;
        font-weight:bold;
        padding:12px;
    ">

        🚑 EMERGENCY AMBULANCE ROUTE FINDER

    </div>

    <div style="
        text-align:center;
        color:#666;
        font-size:15px;
        margin-bottom:10px;
    ">

        Artificial Intelligence Search Algorithms

    </div>

    ""
)

# =====
# LOCATION SECTION
# =====

start_html = widgets.HTML(
    value=""

    <div style="
        text-align:center;
        padding:8px 50px;
    ">

        <b>START LOCATION</b><br>

        <span style="font-size:18px;">
            🏥 Hospital
        </span>

    </div>

    ""
)

destination_html = widgets.HTML(
    value=""

    <div style="
        text-align:center;
        padding:8px 50px;
    ">

        <b>DESTINATION</b><br>

        <span style="font-size:18px;">
            🚑 Patient
        </span>

    </div>

    ""
)

location_section = widgets.HBox(

```

```

        start_html,
        destination_html
    ],

    layout=widgets.Layout(
        justify_content="center",
        align_items="center"
    )
)

# =====
# BUTTON CREATOR
# =====

def make_button(
    text,
    style="primary"
):

    return widgets.Button(

        description=text,

        button_style=style,

        layout=widgets.Layout(
            width="315px",
            height="42px",
            margin="4px"
        )
    )

# =====
# UNINFORMED SEARCH BUTTONS
# =====

bfs_button = make_button(
    "1. Breadth First Search (BFS)"
)

dfs_button = make_button(
    "2. Depth First Search (DFS)"
)

dls_button = make_button(
    "3. Depth Limited Search (DLS)"
)

ids_button = make_button(
    "4. Iterative Deepening Search (IDS)"
)

bidirectional_button = make_button(
    "5. Bidirectional Search"
)

# =====
# INFORMED SEARCH BUTTONS
# =====

greedy_button = make_button(
    "6. Greedy Best First Search"
)

astar_button = make_button(
    "7. A* Search"
)

# =====
# LOCAL SEARCH BUTTONS
# =====

hill_button = make_button(
    "8. Hill Climbing"
)

```

```

genetic_button = make_button(
    "9. Genetic Algorithm"
)

# =====
# OTHER BUTTONS
# =====

compare_button = make_button(
    "10. Compare All Algorithms"
)

map_button = make_button(
    "11. Display City Map"
)

exit_button = make_button(
    "12. Exit",
    "danger"
)

# =====
# DLS DEPTH
# =====

depth_label = widgets.HTML(
    value="<b>DLS Depth Limit:</b>"
)

depth_input = widgets.IntText(
    value=8,
    min=0,
    max=30,
    step=1,

    layout=widgets.Layout(
        width="80px"
    )
)

depth_box = widgets.HBox(

    [
        depth_label,
        depth_input
    ],

    layout=widgets.Layout(
        justify_content="center",
        align_items="center",
        margin="10px"
    )
)

# =====
# UNINFORMED SEARCH SECTION
# =====

uninformed_title = widgets.HTML(

    value=""

    <div style="
        font-size:19px;
        font-weight:bold;
        color:#166534;
    ">

         UNINFORMED SEARCH

    </div>

    <div style="
        color:#555;
        font-size:13px;
        margin-bottom:8px;
    ">

```

```

        Search without heuristic information

</div>

"""
)

uninformed_row1 = widgets.HBox(

    [
        bfs_button,
        dfs_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

uninformed_row2 = widgets.HBox(

    [
        dls_button,
        ids_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

uninformed_row3 = widgets.HBox(

    [
        bidirectional_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

uninformed_section = widgets.VBox(

    [
        uninformed_title,
        uninformed_row1,
        uninformed_row2,
        uninformed_row3,
        depth_box
    ],

    layout=widgets.Layout(
        border="2px solid #86efac",
        padding="12px",
        margin="8px"
    )
)

# =====
# INFORMED SEARCH SECTION
# =====

informed_title = widgets.HTML(

    value="""

<div style="
    font-size:19px;
    font-weight:bold;
    color:#1d4ed8;

```

```

">

     INFORMED SEARCH

</div>

<div style="
    color:#555;
    font-size:13px;
    margin-bottom:8px;
">

    Search using heuristic information

</div>

""

)

informed_row = widgets.HBox(

    [
        greedy_button,
        astar_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )

)

informed_section = widgets.VBox(

    [
        informed_title,
        informed_row
    ],

    layout=widgets.Layout(
        border="2px solid #93c5fd",
        padding="12px",
        margin="8px"
    )

)

# =====
# LOCAL SEARCH SECTION
# =====

local_title = widgets.HTML(

    value=""

    <div style="
        font-size:19px;
        font-weight:bold;
        color:#92400e;
    ">

         LOCAL SEARCH

    </div>

    <div style="
        color:#555;
        font-size:13px;
        margin-bottom:8px;
    ">

        Optimization-based search

    </div>

    ""


```

```

)

local_row = widgets.HBox(

    [
        hill_button,
        genetic_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

local_section = widgets.VBox(

    [
        local_title,
        local_row
    ],

    layout=widgets.Layout(
        border="2px solid #fcd34d",
        padding="12px",
        margin="8px"
    )
)

# =====
# OTHER SECTION
# =====

other_title = widgets.HTML(

    value=""

    <div style="
        font-size:19px;
        font-weight:bold;
    ">

        ⚙ OTHER OPTIONS

    </div>

    ""

)

other_row1 = widgets.HBox(

    [
        compare_button,
        map_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

other_row2 = widgets.HBox(

    [
        exit_button
    ],

    layout=widgets.Layout(
        justify_content="center"
    )
)

other_section = widgets.VBox(

```

```

[
    other_title,
    other_row1,
    other_row2
],

layout=widgets.Layout(
    border="1px solid #cccccc",
    padding="12px",
    margin="8px"
)
)

# =====
# RESULT SECTION
# =====

result_title = widgets.HTML(

    value=""

    <div style="
        font-size:19px;
        font-weight:bold;
        padding-bottom:10px;
    ">

        🏠 RESULT

    </div>

    ""

)

# =====
# NORMAL SEARCH RESULT
# =====

def show_result(
    name,
    path,
    visited
):

    with output:

        clear_output(
            wait=True
        )

        print("=" * 80)

        print(
            f"                {name}"
        )

        print("=" * 80)

        if path:

            print("\n✅ PATH FOUND\n")

            print(
                " → ".join(path)
            )

            print()

            print(
                "Path Cost      :",
                path_cost(path),
                "km"
            )

            print(
                "Nodes Visited :",

```

```

        len(visited)
    )

    print(
        "\nVisited Order:"
    )

    print(
        " → ".join(visited)
    )

else:

    print(
        "\n❌ NO PATH FOUND"
    )

    print(
        "\nVisited Order:"
    )

    print(
        " → ".join(visited)
    )

# =====
# HILL CLIMBING RESULT
# =====

def show_hill_result():

    best_route, history = hill_climbing(
        START,
        GOAL
    )

    with output:

        clear_output(
            wait=True
        )

        print("=" * 80)

        print(
            "
                                HILL CLIMBING"
        )

        print("=" * 80)

        if best_route is None:

            print(
                "\n❌ No valid route found."
            )

            return

        print(
            "\nRoute Improvement:\n"
        )

        for i, route in enumerate(
            history,
            start=1
        ):

            print(
                f"Step {i}: "
                f"{ ' → '.join(route)}"
            )

            print(
                f"
                                Cost = "
                f"{safe_path_cost(route)} km"
            )

```

```

print("\n" + "-" * 80)

print(
    "Final Route:"
)

print(
    " → ".join(best_route)
)

print()

print(
    "Final Path Cost:",
    safe_path_cost(best_route),
    "km"
)

print(
    "Improvements:",
    len(history) - 1
)

# =====
# GENETIC ALGORITHM RESULT
# =====

def show_genetic_result():

    best_route, best_cost, progress = \
        genetic_algorithm(
            START,
            GOAL,
            population_size=20,
            generations=40,
            mutation_rate=0.20,
            seed=42
        )

with output:

    clear_output(
        wait=True
    )

    print("=" * 80)

    print(
        "                                GENETIC ALGORITHM"
    )

    print("=" * 80)

    if best_route is None:

        print(
            "\n❌ No valid route found."
        )

        return

    print(
        "\nGeneration Progress:\n"
    )

    for generation, route, cost in progress:

        if (
            generation <= 5
            or generation % 5 == 0
            or generation == 40
        ):

            print(
                f"Generation {generation:2d}"
                f" : {cost} km"
            )

```

```

        print("\n" + "-" * 80)

        print(
            "Best Route:"
        )

        print(
            " → ".join(best_route)
        )

        print()

        print(
            "Minimum Path Cost:",
            best_cost,
            "km"
        )

        print(
            "Total Generations:",
            len(progress)
        )

# =====
# BUTTON CALLBACKS
# =====

def run_bfs(button):

    path, visited = bfs(
        START,
        GOAL
    )

    show_result(
        "BREADTH FIRST SEARCH (BFS)",
        path,
        visited
    )

def run_dfs(button):

    path, visited = dfs(
        START,
        GOAL
    )

    show_result(
        "DEPTH FIRST SEARCH (DFS)",
        path,
        visited
    )

def run_dls(button):

    limit = depth_input.value

    path, visited = dls(
        START,
        GOAL,
        limit
    )

    show_result(
        f"DEPTH LIMITED SEARCH (DLS) - LIMIT {limit}",
        path,
        visited
    )

def run_ids(button):

    path, visited = ids(
        START,
        GOAL
    )

```

```

    show_result(
        "ITERATIVE DEEPENING SEARCH (IDS)",
        path,
        visited
    )

def run_bidirectional(button):

    path, visited = bidirectional_search(
        START,
        GOAL
    )

    show_result(
        "BIDIRECTIONAL SEARCH",
        path,
        visited
    )

def run_greedy(button):

    path, visited = greedy_best_first_search(
        START,
        GOAL
    )

    show_result(
        "GREEDY BEST FIRST SEARCH",
        path,
        visited
    )

def run_astar(button):

    path, visited = a_star(
        START,
        GOAL
    )

    show_result(
        "A* SEARCH",
        path,
        visited
    )

# =====
# COMPARE ALL ALGORITHMS
# =====

def compare_all(button):

    # -----
    # Traditional algorithms
    # -----

    bfs_path, bfs_visited = bfs(
        START,
        GOAL
    )

    dfs_path, dfs_visited = dfs(
        START,
        GOAL
    )

    dls_path, dls_visited = dls(
        START,
        GOAL,
        depth_input.value
    )

    ids_path, ids_visited = ids(
        START,
        GOAL
    )

    bidirectional_path, bidirectional_visited = \
        bidirectional_search(

```

```

        START,
        GOAL
    )

    greedy_path, greedy_visited = \
        greedy_best_first_search(
            START,
            GOAL
        )

    astar_path, astar_visited = \
        a_star(
            START,
            GOAL
        )

# -----
# Hill Climbing
# -----

hill_path, hill_history = hill_climbing(
    START,
    GOAL
)

# -----
# Genetic Algorithm
# -----

ga_path, ga_cost, ga_progress = \
    genetic_algorithm(
        START,
        GOAL,
        population_size=20,
        generations=40,
        mutation_rate=0.20,
        seed=42
    )

results = [

    (
        "BFS",
        "Uninformed",
        bfs_path,
        len(bfs_visited),
        "Nodes"
    ),

    (
        "DFS",
        "Uninformed",
        dfs_path,
        len(dfs_visited),
        "Nodes"
    ),

    (
        "DLS",
        "Uninformed",
        dls_path,
        len(dls_visited),
        "Nodes"
    ),

    (
        "IDS",
        "Uninformed",
        ids_path,
        len(ids_visited),
        "Nodes"
    ),

    (
        "Bidirectional",
        "Uninformed",
        bidirectional_path,
        len(bidirectional_visited),
        "Nodes"
    ),

]

```

```

(
    "Greedy Best First",
    "Informed",
    greedy_path,
    len(greedy_visited),
    "Nodes"
),

(
    "A*",
    "Informed",
    astar_path,
    len(astar_visited),
    "Nodes"
),

(
    "Hill Climbing",
    "Local Search",
    hill_path,
    len(hill_history) - 1,
    "Improvements"
),

(
    "Genetic Algorithm",
    "Local Search",
    ga_path,
    len(ga_progress),
    "Generations"
)
]

```

with output:

```

clear_output(
    wait=True
)

print("=" * 125)

print(
    "
                                ALGORITHM COMPARISON"
)

print("=" * 125)

print()

print(
    f"{'Algorithm':<23}"
    f"{'Category':<17}"
    f"{'Path Cost':<14}"
    f"{'Detail':<15}"
    f"Path"
)

print("-" * 125)

for (
    name,
    category,
    path,
    detail,
    detail_name
) in results:

    if path is not None:

        cost = safe_path_cost(
            path
        )

        path_text = (
            " → ".join(path)
        )

```

```

        print(
            f"{name:<23}"
            f"{category:<17}"
            f"{str(cost) + ' km':<14}"
            f"{str(detail) + ' ' + detail_name:<15}"
            f"{path_text}"
        )

    else:

        print(
            f"{name:<23}"
            f"{category:<17}"
            f"'N/A':<14}"
            f"{str(detail) + ' ' + detail_name:<15}"
            f"No Path"
        )

print("-" * 125)

print()

# Find mathematically shortest route in the graph

actual_shortest = min(
    ALL_VALID_PATHS,
    key=safe_path_cost
)

print(
    "📍 Start:",
    START
)

print(
    "🎯 Goal :",
    GOAL
)

print(
    "🛣️ Shortest route in this graph:"
)

print(
    " → ".join(actual_shortest)
)

print(
    "Distance:",
    safe_path_cost(actual_shortest),
    "km"
)

# =====
# DISPLAY CITY MAP
# =====

def display_city_map(button):

    with output:

        clear_output(
            wait=True
        )

        print("=" * 80)

        print(
            "
                                CITY ROAD MAP"
        )

        print("=" * 80)

        print()

        print(
            "🏥 Hospital"
        )

        print(

```

```

    " |"

)

print(
    " |— A — D — H — K — L — M — N — 🏠 Patient"
)

print(
    " | | \ | | | |"
)

print(
    " |— B — E — I — J —————|"
)

print(
    " | | |"
)

print(
    " |— C — F — G —|"
)

print()

print("=" * 80)

print(
    "ROAD CONNECTIONS"
)

print("-" * 80)

displayed = set()

for node in graph:
    for neighbor, distance in graph[node].items():
        road = tuple(
            sorted(
                [node, neighbor]
            )
        )

        if road not in displayed:
            print(
                f"{node:<12}"
                f" ---- {distance} km ---- "
                f"{neighbor}"
            )

            displayed.add(road)

print()

print(
    "Total Locations:",
    len(graph)
)

print(
    "Total Valid Routes:",
    len(ALL_VALID_PATHS)
)

actual_shortest = min(
    ALL_VALID_PATHS,
    key=safe_path_cost
)

print()

print(
    "Shortest Route:"
)

print(
    " → ".join(actual_shortest)
)

```

```

    )

    print(
        "Distance:",
        safe_path_cost(actual_shortest),
        "km"
    )

# =====
# EXIT
# =====

def exit_application(button):

    with output:

        clear_output(
            wait=True
        )

        print("=" * 80)

        print(
            "
            🚑 EMERGENCY AMBULANCE ROUTE FINDER"
        )

        print("=" * 80)

        print()

        print(
            "Application closed."
        )

        print()

        print(
            "Run the cell again to restart."
        )

# =====
# CONNECT BUTTONS
# =====

bfs_button.on_click(
    run_bfs
)

dfs_button.on_click(
    run_dfs
)

dls_button.on_click(
    run_dls
)

ids_button.on_click(
    run_ids
)

bidirectional_button.on_click(
    run_bidirectional
)

greedy_button.on_click(
    run_greedy
)

astar_button.on_click(
    run_astar
)

hill_button.on_click(
    lambda button: show_hill_result()
)

genetic_button.on_click(
    lambda button: show_genetic_result()
)

genetic_button.on_click(

```

```

compare_button.on_click(
    compare_all
)

map_button.on_click(
    display_city_map
)

exit_button.on_click(
    exit_application
)

# =====
# DEFAULT OUTPUT
# =====

with output:

    print("=" * 80)

    print(
        "Select a search algorithm to find the ambulance route."
    )

    print()

    print(
        "Starting Point : Hospital"
    )

    print(
        "Destination      : Patient"
    )

    print()

    print(
        "🔍 Uninformed Search:"
    )

    print(
        "BFS | DFS | DLS | IDS | Bidirectional"
    )

    print()

    print(
        "🎯 Informed Search:"
    )

    print(
        "Greedy Best First | A*"
    )

    print()

    print(
        "🏔️ Local Search:"
    )

    print(
        "Hill Climbing | Genetic Algorithm"
    )

    print()

    print(
        "Valid Routes Available:",
        len(ALL_VALID_PATHS)
    )

    print()

    shortest = min(
        ALL_VALID_PATHS,
        key=safe_path_cost
    )

    print(
        "Shortest Route:",
        " → ".join(shortest)
    )

```

```

    print(
        "Shortest Distance:",
        safe_path_cost(shortest),
        "km"
    )

    print("=" * 80)

# =====
# RESULT SECTION
# =====

result_section = widgets.VBox(

    [
        result_title,
        output
    ],

    layout=widgets.Layout(
        border="1px solid #cccccc",
        padding="15px",
        margin="8px"
    )
)

# =====
# MAIN APPLICATION
# =====

application = widgets.VBox(

    [
        title,

        location_section,

        uninformed_section,

        informed_section,

        local_section,

        other_section,

        result_section
    ],

    layout=widgets.Layout(
        width="100%",
        max_width="1200px",
        margin="auto",
        padding="15px",
        border="2px solid #888888"
    )
)

# =====
# DISPLAY APPLICATION
# =====

display(application)

```



EMERGENCY AMBULANCE ROUTE FINDER

Artificial Intelligence Search Algorithms

START LOCATION



Hospital

DESTINATION



Patient



UNINFORMED SEARCH

Search without heuristic information

1. Breadth First Search (BFS)
2. Depth First Search (DFS)
3. Depth Limited Search (DLS)
4. Iterative Deepening Search (IDS)
5. Bidirectional Search

DLS Depth Limit:

8



INFORMED SEARCH

Search using heuristic information

6. Greedy Best First Search
7. A* Search



LOCAL SEARCH

Optimization-based search

8. Hill Climbing
9. Genetic Algorithm



OTHER OPTIONS

10. Compare All Algorithms
11. Display City Map
12. Exit



RESULT

ALGORITHM COMPARISON

Algorithm	Category	Path Cost	Detail	Path
BFS	Uninformed	33 km	16 Nodes	Hospital → C → G → J → L → M → N → Patient
DFS	Uninformed	52 km	16 Nodes	Hospital → A → D → E → B → F → C → G → I → H → K → J → L → M → N → Patient
DLS	Uninformed	30 km	438 Nodes	Hospital → A → D → H → K → L → M → N → Patient
IDS	Uninformed	33 km	2635 Nodes	Hospital → C → G → J → L → M → N → Patient
Bidirectional	Uninformed	33 km	10 Nodes	Hospital → C → G → J → L → M → N → Patient
Greedy Best First	Informed	33 km	8 Nodes	Hospital → C → G → J → L → M → N → Patient
A*	Informed	30 km	13 Nodes	Hospital → B → E → I → J → L → M → N → Patient
Hill Climbing	Local Search	30 km	1 Improvements	Hospital → B → E → H → K → L → M → N → Patient
Genetic Algorithm	Local Search	30 km	40 Generations	Hospital → B → E → H → K → L → M → N → Patient



Start: Hospital



Goal : Patient

Github Link

https://github.com/Samu7709/AIML_Lab_Assignments_CA